TECHNICAL REPORT

Binary Hierarchical ORAM for Interactive Two-Party Computation

Large routing buckets, small lookup buckets, and packed payloads

17 October 2026 · Semi-honest two-server setting

Abstract

We adapt the hierarchical design of Microrama to streamed, interactive two-party computation. A read invalidates its old copy and reinserts one batch representative, eliminating key deduplication during rebuilding. The hierarchy is binary, its base is a hash table, and no recursive position map or deamortized schedule is required. Secret PRFs compress application keys to 128 bits and generate all table addresses.

Rebuilding first routes records through 512-slot buckets, then refines each bucket into 64- or 128-slot lookup buckets using recursive ORCompact. A metadata-only planner identifies exact final-bucket excess before entries move. One 64-slot residual buffer per parent collects this excess; a further ORCompact merges all parent buffers into a fixed table stash. Every read searches its selected small bucket and that stash. We include the cost of re-admitting the stash at the next rebuild.

With a batch of 1024, 64-slot lookup buckets use approximately 21.4 and 28.5 MB per lookup at $N = 2^{28}$ and 2^{32}, including rebuilding, reads, initial PRF evaluation, duplicate handling, and packed-payload movement. Rebuilding accounts for about 11.1 and 15.0 MB and is now the larger component. The current Circuit OPRAM planning ledger uses 226.5 and 286.9 MB. These are communication estimates for specified kernels, not measured throughput. Payloads use a concrete XOR-share/word-OT backend; base-OT setup, CPU, initialization, replication, and implementation validation remain separate.

Primary: 512-to-64, $b = 1024$	$N = 2^{28}$	$N = 2^{32}$
Rebuild MB/query	11.147	14.976
Read MB/query	9.439	12.676
Total MB/query	**21.389**	**28.455**
Current Circuit OPRAM MB/query	226.507	286.945
Top global stash slots	112	264

All traffic tables use decimal MB and omit packet framing unless stated. The probability analysis assumes ideal fresh PRFs and treats 128-bit fingerprint collisions separately. Its numerical evaluations are not interval-certified proofs.

Contents

1 Scope, result, and assumptions **3**

2 Hierarchy, records, and the binary rebuild schedule **3**
2.1 Mutable tables and unique occupied copies . . . 3
2.2 One hash-table base and one batch deduplication . . . 4
2.3 Direct binary-counter rebuilds . . . 4

3 PRFs and the mixed representation **4**
3.1 All addresses come from secret PRFs . . . 4
3.2 Boolean metadata and packed value carriers . . . 5
3.3 A 160-byte secret word operation . . . 5

4 A complete rebuild with local refinement **6**
4.1 Source compaction includes old stashes . . . 6
4.2 The 512-slot butterfly . . . 6
4.3 Select exact leaf excess using only metadata . . . 6
4.4 One local residual stash and recursive ORCompact . . . 7
4.5 Merge all parent residuals . . . 7

5 Lookup, deletion, and batch parallelism **7**

6 Overflow and fingerprint error budgets **8**
6.1 512-slot routing buckets . . . 8
6.2 A 64-slot local residual buffer . . . 8
6.3 The merged table stash . . . 9
6.4 Finite fingerprints and implementation qualification . . . 9

7 Circuit and communication accounting **10**
7.1 ORCompact and exact-padding planners . . . 10
7.2 Local refinement costs . . . 10
7.3 Read and frontend ledger . . . 11

8 Numerical results and sensitivities **11**
8.1 Primary traffic comparison . . . 11
8.2 Rebuild decomposition . . . 12
8.3 Batch-size sensitivity . . . 12
8.4 Packed operation sensitivity . . . 13

9 Space, parallel work, and latency **13**
9.1 Persistent state and rebuilding workspace . . . 13
9.2 Mixed-protocol depth . . . 13
9.3 Replicas and throughput . . . 14
9.4 An optional parent-local persistent stash . . . 14

10 Validation, remaining work, and recommendation **14**

A Planner derivation and finite counts **15**

B Reproduction and source provenance **15**

1 Scope, result, and assumptions

This report specifies a private dictionary for fixed-length 32-byte keys and 32-byte values. Garbler G and evaluator E are noncolluding servers; a static semi-honest adversary corrupts at most one of them. A query arrives as two XOR shares of its application key. The answer and its presence bit remain shared. Database contents are similarly shared or encoded during initialization. The main workload is a fixed dictionary with application reads; each read still mutates the ORAM representation. The reference service rate is 10,000 lookups/s, but no particular deployment is assumed to achieve it.

Our principal conclusion is a communication feasibility result. Refining large routing buckets removes much of the earlier read-scan overhead while adding modest amortized work to rebuilding. A 64-slot leaf performs best in the evaluated average-traffic cases. A 128-slot leaf has a much smaller largest-table stash, which can matter for latency and large-table contention. The comparison is against the current Circuit-OPRAM-inspired compilation ledger, not a lower bound for all Circuit OPRAM implementations.

Symbol	Meaning
N	Dictionary capacity, 2^{28} or 2^{32} genuine records
b	Request batch size; primary choice 1024
$n_i = b2^i$	Logical capacity of lower level i
$L = \log_2(N/b)$	Number of lower levels; permanent top has capacity N
$Z = 512$	Routing-bucket capacity; nominal mean occupancy 256
$z \in \{64, 128\}$	Lookup-bucket capacity; nominal mean occupancy $z/2$
$d = \log_2(Z/z)$	Additional PRF bits for the local refinement
$s = 64$	Temporary residual capacity per 512-slot parent
$G_z(n)$	Fixed capacity of the merged residual stash of an n-record table
$Q = 2^{32}$	Access horizon used for capacity calculations
$c_V = 160$	Bytes per secret swap or product of a complete 256-bit payload

Public information includes capacities, batch size, level occupancy, rebuild times, selected physical bucket indices, and message sizes. Keys, record occupancy, absence markers, hit locations, routing controls, and residual counts remain private. All operations have fixed public capacity. Failure events are explicitly bounded; an implementation must handle an overflow without silently discarding a valid entry.

2 Hierarchy, records, and the binary rebuild schedule

2.1 Mutable tables and unique occupied copies

Lower levels have capacities $b, 2b, \ldots, N/2$, with a permanent top of capacity N. A table of logical capacity n has $2n$ main slots. After refinement these slots form $2n/z$ lookup buckets. A separate $G_z(n)$-slot stash holds excess records. Unlike Microrama's read-only tables, access clears an old occupied bit before reinsertion [1].

An entry consists of a 128-bit canonical fingerprint, two Boolean flags, and a packed 256-bit value. The flags are distinct:

- $\text{occ} = 0$ denotes a dummy or an invalidated old copy.
- $\text{occ} = 1, \text{present} = 1$ denotes a present dictionary record.
- $\text{occ} = 1, \text{present} = 0$ denotes a keyed cached absence, also called a tombstone.

The steady metadata width is therefore $R = 130$ bits. Routing tags exist only during rebuilding. The construction maintains one occupied copy of each represented fingerprint across main buckets and stashes.

On a hit, including a cached absence, clear the old occupied bit and carry the entry into the new batch. On a miss, create a keyed cached absence. Repeated misses must find that absence before returning to older table hashes. Inserting an unkeyed dummy on a miss would reveal repeated queries. Only the full top refresh can discard cached absences: it retires all old hashes and empties every lower level. Application deletion and concurrent writes require an additional functionality specification.

2.2 One hash-table base and one batch deduplication

The base is a hash table of logical capacity b, not a dense online stash. For $b = 1024$ it initially has four 512-slot routing buckets and ends with 32 buckets of 64 slots or 16 buckets of 128 slots, plus its residual stash.

At batch entry, compute canonical fingerprints, bitonically sort them, and privately mark one representative per distinct key. Duplicate request slots remain in the fixed batch and issue dummy probes. Representatives alone are reinserted as occupied entries. At batch exit, a segmented propagation copies the representative's answer through its equal-key run, then saved sort controls restore original request order. None of these controls is opened. This is a one-time batch cost; there is no position-map recursion.

2.3 Direct binary-counter rebuilds

After a batch, rebuild directly into the smallest empty lower level, merging the new batch and every occupied smaller level. If all lower levels are full, refresh the top together with all lower levels. Intermediate carry tables need not be materialized. The schedule depends on batch count even when a batch has fewer than b distinct keys.

Over a full counter cycle, lower capacity n_i is the destination once per $2n_i$ accesses; the top is rebuilt once per N accesses. Consequently,

$$\overline{C} = \sum_{i=0}^{L-1} \frac{C(n_i)}{2n_i} + \frac{C_{\text{top}}(N)}{N}. \tag{1}$$

Each lower table is present half the time. The mean number of searched tables is $L/2 + 1$, while the maximum is $L + 1$: 10/19 at $N = 2^{28}$ and 12/23 at $N = 2^{32}$ for $b = 1024$.

Independent ORAM replicas can service requests while one replica rebuilds. This avoids requiring a deamortized construction, but does not remove amortized rebuilding work or the storage cost of replicas. Static application data allow replicas to evolve independently; application writes require a separate consistency protocol.

3 PRFs and the mixed representation

3.1 All addresses come from secret PRFs

Compute the 16-byte canonical key using AES-CMAC under a jointly secret key. For a fixed 32-byte application key, this uses two AES encryptions. Use fresh independent secret contexts for each published table epoch's real and dummy addresses. A table's AES output supplies both its parent-bucket bits and the d refinement bits. This is one PRF evaluation per retained rebuild slot, not one evaluation per routing stage. Dummy-slot evaluations are also charged to preserve the fixed circuit schedule.

We use the NIST-listed 5120-AND AES encryption datapath and 1280-AND key schedule [3]. Thus initial key compression costs 10,240 ANDs per query. The shared CMAC subkey and expanded-key

setup cost 6400 ANDs once per initialized state. A fresh table's two key schedules are charged at 2560 ANDs, with two 128-bit shared key inputs. All Boolean ANDs cost 32 bytes in the half-gate ledger [4]. XORs contribute CPU work but no garbled-table bytes.

For a read, the ledger computes both a true address and a fresh dummy address, then privately selects between them. The dummy PRF input contains a unique epoch/request identifier. Neither party knows the unused dummy address in clear, preventing identification of the hit level by comparison. The next table's selected index is disclosed only after updating the hidden found flag. Requested bucket indices are visible; fingerprints and full routing tags remain secret.

3.2 Boolean metadata and packed value carriers

Metadata stays in ordinary Boolean GC representation. Each party retains 16 bytes per encoded metadata bit. A 256-bit payload is represented as ordinary XOR shares $V = V_G \oplus V_E$, each share occupying two 128-bit words. This is a precise packed-value interface; it is not an assertion that two ordinary Boolean wire labels encode a 256-bit value.

Payloads stay in this representation throughout compaction, routing, selection, and result propagation. No share-to-GC re-input is needed for pure movement. A later application that computes on the returned value must pay for its selected representation boundary. Initialization must establish these payload shares; conversion from a different preexisting opaque encoding is not free.

3.3 A 160-byte secret word operation

Let the GC output control bit c have zero-label color c_G and active-label color c_E. With the standard Free-XOR offset satisfying $\mathrm{color}(\Delta) = 1$, these give local XOR shares $c = c_G \oplus c_E$, with no network conversion. For payload shares x_P, y_P, define $d_P = x_P \oplus y_P$. Use the two word OTs below with fresh independent random 256-bit masks:

$$\begin{aligned} &G \longrightarrow E : (r_G,\ r_G \oplus d_G), \quad \text{choice } c_E, \quad \text{received } u_E; \\ &E \longrightarrow G : (r_E,\ r_E \oplus d_E), \quad \text{choice } c_G, \quad \text{received } u_G. \end{aligned}$$

Set

$$\begin{aligned} t_G &= c_G d_G \oplus r_G \oplus u_G, & t_E &= c_E d_E \oplus r_E \oplus u_E, \\ x'_P &= x_P \oplus t_P, & y'_P &= y_P \oplus t_P. \end{aligned}$$

Then $t_G \oplus t_E = c(x \oplus y)$, so the outputs are a conditional swap. Replacing d by a single word gives a secret bit-times-word product, used for one-hot read extraction.

With chosen-message OT extension, each directed OT budgets two 32-byte ciphertexts and 16 bytes of extension matrix communication at security parameter 128: 80 bytes per direction, 160 bytes total [5]. Base OTs are amortized separately. The two directions have independent extension instances and domain-separated pad hashes. Fresh masks hide each output share. The circuit controls remain shared, and no routing schedule is opened. This explicit semi-honest construction requires implementation validation; malicious security would require additional checks.

4 A complete rebuild with local refinement

4.1 Source compaction includes old stashes

A lower rebuild into capacity n initially has $2n$ *main-array* source slots: all smaller main tables plus the new batch padded to $2b$. A top rebuild has $4N$ main-array source slots. Old global stashes add physical slots beyond these counts and cannot be ignored.

We use two compactions. First compact the main arrays by occupied bits and keep the first n slots. At the top, first clear occ for cached absences. There are at most n occupied retained keys in all sources combined, so keeping this main-array prefix is safe. Append every source stash at fixed public offsets after slot n, clear absent records there at the top, and pad the combined array to $2n$. A second ORCompact keeps its first n slots. The sum of source stash capacities is at most n in every evaluated configuration, and the model checks this assertion. The base has no old stash and needs only the first compaction.

This extra pass avoids a hidden source-size increase or a free stash merge. All old tags can be discarded before these compactions. Next compute the new PRF tags for the n retained slots and distribute these slots publicly into $2n/Z$ parents with $Z/2$ designated input positions each, padding the rest. Every initial parent has at most $Z/2$ occupied slots.

4.2 The 512-slot butterfly

Let $p = 2n/Z = 2^h$. A binary butterfly routes records by h parent-index bits. It has h stages, each containing $p/2$ independent merge-splits. Every merge-split acts on $2Z = 1024$ slots. It marks real entries for the left output and enough dummies to make exactly Z marks, then invokes ORCompact [2]. This works unless a real side exceeds Z; Section 6 bounds that event.

The d extra leaf-index bits travel through this network too. They are not used to choose a 512-slot parent but are needed for local refinement. Dropping them and silently recomputing them would omit another PRF cost. After global routing, the parent-index bits can be discarded, leaving 130 metadata bits plus d suffix bits per entry.

4.3 Select exact leaf excess using only metadata

A naive shrinking binary split can eject records at an intermediate node even when their final leaf has room. We instead determine final leaf excess first, using one-bit planners, and then recursively split a feasible array. For one 512-slot parent, let $g = 512/z = 2^d$ and let X_j count occupied records whose suffix names leaf j.

For every leaf j, perform the following fixed procedure:

1. Form $M_j[i] = \mathrm{occ}_i \wedge [\mathrm{suffix}_i = j]$ for all 512 slots.
2. Compute the ORCompact routing controls for M_j.
3. Run those controls *backwards* on the public one-bit vector $1^z 0^{512-z}$. This maps the first z output positions back to selected input positions.
4. AND the inverse mask with M_j. This keeps exactly $\min(X_j, z)$ marked entries of leaf j.

If $X_j < z$, the inverse mask also contains unmarked slots; the last AND removes them. ORCompact is stable on marked entries, although any fixed private choice of z entries would suffice. Invert the keep mask within M_j to obtain that leaf's excess mask. These masks are disjoint across leaves, so their union

is a free XOR. The final residual count is exactly

$$r = \sum_{j=0}^{g-1} (X_j - z)_+. \tag{2}$$

Only one-bit arrays move during this selection step. Neither keys nor payloads are routed separately for each leaf.

4.4 One local residual stash and recursive ORCompact

ORCompact the complete 512-slot parent once using the residual mask. The first r records are residuals. Copy the first 64 slots' data into a local stash at fixed public offsets, set its occupied bits to $[i < r]$, and set the main occupied bits to $\mathrm{occ}_i \wedge \neg[i < r]$ for $0 \le i < 64$. Slots $r, \ldots, 63$ keep their original occupied bits; the remaining main slots are also unchanged. The copies are unconditional and local; no conditional payload move is added. Preserve the presence bit in the stash. Check $r > 64$ privately as an overflow condition.

Every remaining leaf now has at most z occupied records. Recursively partition the main 512 slots into two 256-slot arrays, then four 128-slot arrays, and, for $z = 64$, eight 64-slot arrays. Each node uses the same exact-padding ORCompact split. No intermediate node can overflow: its occupied count is at most the sum of the capacities of its descendant leaves. Consumed suffix bits may be dropped after each stage.

This construction uses *one* shared local stash per original parent, not a stash per split node. Its capacity depends on final excess only. In the worst deterministic input, 512 keys can name one leaf and require $512 - z$ residual slots. The choice of 64 is probabilistic and is justified over the access horizon below.

4.5 Merge all parent residuals

There are $p = 2n/512$ parents. Concatenate their fixed 64-slot buffers and ORCompact the resulting

$$p \cdot 64 = n/4 \tag{3}$$

slots by occupied bits. Keep the first $G_z(n)$ slots as the table's persistent global stash, preserving all 130 metadata bits and packed payload shares. Check whether more occupied entries remain. Because $n/4$ is already a power of two, this collector needs no extra padding. The local buffers are temporary and can be reclaimed after publication.

The table now consists of $2n/z$ small lookup buckets and one global stash. The total occupied count remains at most n. Entry uniqueness holds across both parts. No residual count, destination, or compaction control is revealed.

5 Lookup, deletion, and batch parallelism

For each occupied table, first select the request's true or dummy index using the hidden found flag. Reveal only that selected index. The evaluator requests a fresh circuit for the corresponding z-slot bucket and the table's fixed $G_z(n)$-slot stash. Both parties already know the table version and the public set of requested indices.

Inside the circuit, compare the 128-bit fingerprint with every occupied candidate, gated by active $=$ representative $\wedge$ $\neg$found. A dummy physical probe therefore cannot delete an occupied record even if its stored key happens to match. A present record returns its value; a cached absence stops further

real probes and returns absent. Compute all bit-times-payload products in parallel and XOR their output shares. No sequential chain of payload OTs is required for scanning a bucket or stash. Clear the occupied bit of each matched entry. Unique occupied copies and request deduplication make the matching requests disjoint.

All requests to the same public bucket must use one combined circuit with aggregated deletion masks, or serial fresh circuits linked through the updated state. The global stash is shared by *all* requests to that table, so its updates must likewise be combined. Independent circuits reading the same old stash version could restore records deleted by another request. The model retains $bG_z(n)$ stash comparisons per batch and does not assume this coordination removes them.

Garbled tables, gate tweaks, and mutable output labels must be fresh for each invocation/version. On-demand garbling solves the scheduling problem of knowing which bucket circuits are needed. It does not authorize reusing a one-time garbled circuit on changing bucket contents.

Queries and buckets within one table stage parallelize, as do the merge-splits within a rebuild stage. A single query's table probes retain a dependency: the next disclosed index depends on whether it has already found its key. Computing candidate PRFs ahead of time does not allow all real indices to be disclosed at once. This limitation is also present in Microrama's lookup description [1].

6 Overflow and fingerprint error budgets

6.1 512-slot routing buckets

After balanced initial placement, an intermediate bucket receives a sum of fresh-PRF routing indicators with mean at most $Z/2$. Capping previous buckets only removes records, so a first-overflow argument compares with the uncapped network. Chernoff gives

$$\Pr[X \geq Z] \leq (e/4)^{Z/2}. \tag{4}$$

At $Z = 512$ this is at most $2^{-142.67}$ per bucket-stage. Multiply by the public number of bucket-stage tests over the horizon. For the primary cases the resulting bounds are below 2^{-111}.

6.2 A 64-slot local residual buffer

For a parent with $g = 512/z$ children, let $R = \sum_j (X_j - z)_+$. If $R > s$, some subset of a overflowing leaves jointly contains at least $u_a = az + s + 1$ records. Before conditioning on the parent load, its mean is at most $\mu_a = az/2$. A subset union bound gives

$$\Pr[R > s,\ X_{\text{parent}} \leq 512] \leq \sum_{\substack{1 \leq a \leq g \\ u_a \leq 512}} \binom{g}{a} \exp\{-u_a \ln(u_a/\mu_a) + u_a - \mu_a\}. \tag{5}$$

Terms with $u_a > 512$ are impossible when the parent routing succeeds. This is not a claim conditioned on an adversarially fixed full parent; it is a joint event bound under the fresh hash distribution. Parent-routing failure is budgeted separately.

The number of parent refinements over Q accesses is

$$H = \frac{Q(L+2)}{512}. \tag{6}$$

Using $s = 64$, Equation (5) and a union bound yield lifetime probabilities below 2^{-87} for 64-slot leaves and below 2^{-91} for 128-slot leaves in both primary cases.

6.3 The merged table stash

For a table of capacity n, let $M = 2n/z$ and $Y \sim \mathrm{Pois}(z/2)$. The exact moment generating function of a leaf's excess is

$$\begin{aligned} f_z(t) &= \mathbb{E}[e^{t(Y-z)_+}] \\ &= \Pr[Y \le z] + e^{-tz+(z/2)(e^t-1)} \Pr[\mathrm{Pois}((z/2)e^t) > z]. \end{aligned} \tag{7}$$

Under Poissonization the bucket occupancies are independent. Overflow is monotone in the total number of balls, and $\Pr[\mathrm{Pois}(n) \ge n] \ge 1/2$ for integer n. Coupling with exactly n balls therefore gives

$$\Pr[R_{\mathrm{table}} > G] \le 2 \inf_{t>0} \exp[-t(G+1)]\, f_z(t)^M. \tag{8}$$

Fewer occupied records only decrease the event. This argument does not assume independence between levels or rebuilds.

There are $Q/b = 2^{22}$ rebuilds at $b = 1024$. Choose each $G_z(n)$ so that Equation (8) is below 2^{-88}, then round up to a multiple of eight. A union bound gives a global-stash lifetime budget below 2^{-66}. The actual evaluated sums are smaller. The optimizer only chooses a positive Chernoff witness t; evaluating any such witness already gives a valid analytic bound.

$\log_2 n$	$G_{64}(n)$	$G_{128}(n)$	$\log_2 n$	$G_{64}(n)$	$G_{128}(n)$
10	56	56	22	72	56
11	56	56	23	80	56
12	56	56	24	80	56
13	56	56	25	88	64
14	56	56	26	96	64
15	64	56	27	104	64
16	64	56	28	112	64
17	64	56	29	128	64
18	64	56	30	160	64
19	64	56	31	200	64
20	72	56	32	264	64
21	72	56	–	–	–

At the top, the Poisson model predicts about 3.20/51.22 residual records for 64-slot leaves at $N = 2^{28}/2^{32}$. For 128-slot leaves the corresponding expectations are approximately $5.1 \cdot 10^{-6}/8.2 \cdot 10^{-5}$. Expected size does not determine a safe capacity: rare large excesses still require a stash. In particular, omitting residual handling for 128-slot leaves does not meet the stated lifetime target.

6.4 Finite fingerprints and implementation qualification

The above are routing-capacity bounds under ideal PRFs and distinct fingerprints. With D distinct application keys, 128-bit compression has the elementary collision bound $\binom{D}{2}/2^{128}$. For $N = 2^{32}$ and another 2^{32} distinct queried keys, $D \le 2^{33}$ gives a bound near 2^{-63}. The routing bounds therefore do not establish a combined 2^{-64} failure guarantee for this maximum workload. A 128-bit label length also does not imply a 128-bit service-lifetime error guarantee.

Fresh table keys are chosen after the source record set is fixed. This supports a conditional analysis for adaptive access histories; a complete reduction still needs to specify exposure of truncated outputs, the PRF instantiation, initialization, and the canonical-key collision event.

TODO. If a strict total lifetime target is required, allocate its budget between fingerprint collisions, finite-PRF advantages, and capacity failures; consider a wider fingerprint or an explicit collision-resolution mechanism.

TODO. Certify the numerical Chernoff witnesses with interval arithmetic. The present calculation uses stable floating-point tail and log-MGF formulas and rounds capacities upward, but is not machine-checked analysis.

7 Circuit and communication accounting

7.1 ORCompact and exact-padding planners

For $a = 2^k$ slots, ORCompact has exactly

$$S(a) = \frac{a}{2} \log_2 a \tag{9}$$

conditional swaps [2]. A constructive Boolean schedule planner used in this ledger costs at most

$$P(a) = a(k - \tfrac{1}{2}) - k + 1 \tag{10}$$

ANDs. To split a slots evenly, forming exactly $a/2$ marks adds at most $3ka + 4a - 2k - 2$ ANDs, giving

$$A_{\text{split}}(a) = 4ka + \tfrac{7}{2}a - 3k - 1. \tag{11}$$

These Boolean counts are this report's compilation allowances, not gate counts claimed by the ORCompact paper. Appendix A explains the decomposition and the dummy-rank width requirement.

A conditional swap of an r-bit Boolean record costs r ANDs: compute $t = c \wedge (x \oplus y)$ bitwise, then $x' = x \oplus t, y' = y \oplus t$. The complete mixed-record operation therefore costs $32r + c_V$ bytes. Public copies, public permutations, and XOR-share fan-out add no garbled tables, although they do incur memory and CPU work.

For the global butterfly, $p = 2n/512$ and $h = \log_2 p$. Its $ph/2$ calls each have $S(1024) = 5120$ record swaps. The model conservatively carries all $h + d$ routing bits through this global network, for

$$C_{\text{butterfly}} = \frac{nh}{512} \left[32A_{\text{split}}(1024) + 5120\{32(130 + h + d) + c_V\}\right]. \tag{12}$$

The initial PRF cost is $5120n$ ANDs, plus table-key setup. Old source compactions and the global residual collector move only 130 metadata bits.

7.2 Local refinement costs

Each leaf's excess planner costs $512d + P(512) + S(512) + 512$ ANDs: membership flags, schedule planning, one-bit inverse routing, and mask intersection. Multiply by $512/z$. The residual extraction uses $S(512) = 2304$ complete-entry swaps and $P(512) + 197$ planner/check ANDs. Recursive splits cost the sum of the node costs at sizes 512, 256, and optionally 128; already consumed suffix bits are discarded. The resulting kernel is:

Per 512-slot parent	64-slot leaves	128-slot leaves
Excess-selection planner ANDs	69,568	32,736
Residual-extraction planner ANDs	4,541	4,541
Recursive-split planner ANDs	54,362	38,322
Total planner ANDs	128,471	75,599
Metadata-movement ANDs	1,117,952	876,544
Complete payload swaps	8,448	6,656
Payload swap layers	33	26
Total MB at $c_V = 160$	41.237	31.534

For each rebuilt table, multiply these counts by $p = 2n/512$. The global collector adds $P(n/4)$ planner ANDs and $S(n/4)$ swaps of a 130-bit metadata record plus its packed value. The two-stage source admission adds a full second $2n$-slot compaction whenever old stashes exist. Both costs are included before applying Equation (1).

7.3 Read and frontend ledger

One occupied table scans $z + G_z(n)$ slots. A conservative read allowance is

$$A_{\text{read}}(n) = 10240 + 134[z + G_z(n)] + h + d + 4 \tag{13}$$

ANDs, plus $z + G_z(n)$ packed word products at c_V bytes each. The 10240 term charges true and dummy PRFs. The scan coefficient covers fingerprint equality, occupancy/active gating, presence reduction, deletion, and accumulation. Disclosed address bits add a small explicit bidirectional term. Sum lower levels with weight 1/2 and the top with weight one.

For $b = 2^\beta$, a bitonic network has $b\beta(\beta+1)/4$ comparators. The model charges 511 ANDs per comparator for 128-bit comparison, fingerprint swapping, and inverse presence-bit routing, plus one packed result swap on the inverse network. It also reserves 140 ANDs and three packed selectors per adjacent input for a work-efficient parallel segmented propagation. This is deliberately more than the earlier sequential propagation allowance.

The initial 32-byte secret-shared query input costs 16,384 bytes under the existing input convention; AES-CMAC adds 327,680 bytes. No initial database hashing, base-OT setup, client delivery, or application computation on the result is amortized into the steady-state rows.

8 Numerical results and sensitivities

8.1 Primary traffic comparison

N	Construction	Rebuild MB	Read MB	Total MB
2^{28}	512, unsplit	8.459	26.056	35.317
2^{28}	512 to 64	11.147	9.439	21.389
2^{28}	512 to 128	10.731	11.556	23.090
2^{28}	Circuit OPRAM	–	–	226.507
2^{32}	512, unsplit	11.603	31.267	43.673
2^{32}	512 to 64	14.976	12.676	28.455
2^{32}	512 to 128	14.469	13.921	29.193
2^{32}	Circuit OPRAM	–	–	286.945

512-to-64 traffic	$N = 2^{28}$	$N = 2^{32}$
Frontend and dedup MB/query	0.803	0.803
Total with 5% framing MB/query	22.458	29.878
Maximum occupied-level read MB/query	17.766	23.564

All rows use complete application lookups. Our hierarchy directly stores canonical keys and has no recursive position maps or outer pair of cuckoo table probes. The Circuit OPRAM ledger includes both cuckoo probes and its recursive machinery. It also uses ordinary Boolean payload movement in its primary row. These are complete alternative layouts, not identical circuits with only one kernel substituted. Its selected batch is 32; the hierarchical primary batch is 1024. The comparison does not equalize batch-fill delay or measured latency.

Refinement makes rebuilding the larger component for both leaf sizes. The 64-slot choice minimizes average traffic in the primary rows, but at $N = 2^{32}$ its top lookup scans $64 + 264 = 328$ slots, versus $128 + 64 = 192$ for 128-slot leaves. Thus the smaller main bucket is not universally the smaller read operation. Much of the average advantage comes from the smaller tables' more modest stashes.

8.2 Rebuild decomposition

Rebuild component, MB/query	$N = 2^{28}$	$N = 2^{32}$
Routing PRF and key schedules	1.638	1.966
Original source compactions	0.995	1.286
Old-stash re-admission	0.840	1.115
Global butterfly	5.715	8.199
Local excess, extraction, split	1.558	1.870
Global residual collector	0.092	0.123
Packed payload operations	0.309	0.417
Total including fresh-key input	11.147	14.976

The dominant rebuild terms move Boolean metadata through the global butterfly and source compactions. One-bit planners are important but do not make record movement free. Packing the payload does not pack the 128-bit fingerprint or the routing bits used by GC.

8.3 Batch-size sensitivity

b	28, $z64$	28, $z128$	32, $z64$	32, $z128$	Fill ms
512	21.989	23.831	29.073	29.935	25.55
1024	21.389	23.090	28.455	29.193	51.15
2048	20.733	22.347	27.799	28.450	102.35
4096	20.074	21.583	27.122	27.686	204.75

Larger batches reduce the number of levels and the per-lookup rebuild frequency of the smallest tables, while increasing initial sort work and batch-fill time. At rate q, ideal mean fill delay is $(b - 1)/(2q)$: 51.15 ms at $b = 1024$ and 10,000 requests/s. This is only fill delay, before circuit service and queueing. The evaluated sweep is not a full latency-constrained optimization.

8.4 Packed operation sensitivity

c_V (bytes)	28, $z64$	28, $z128$	32, $z64$	32, $z128$
80	21.121	22.790	28.087	28.809
160	21.389	23.090	28.455	29.193
320	21.924	23.691	29.191	29.961
8192	48.263	53.225	65.428	67.740

The 160-byte row corresponds to the specified two-direction word-OT backend. The 80/320-byte rows are parameter sensitivities. The 8192-byte row is an ordinary 256-AND Boolean-payload movement reference, with the same metadata kernels; it is not the storage cost of two packed shares. It illustrates how much of the conclusion survives without a packed movement discount.

9 Space, parallel work, and latency

9.1 Persistent state and rebuilding workspace

Before a full top rebuild, the maximum main-array allocation is $4N - 2b$ slots. Add the public global stash capacity of every table. A packed record uses $16 \cdot 130 + 32 = 2112$ bytes per party. This gives:

512-to-64 resource	$N = 2^{28}$	$N = 2^{32}$
Packed state per party, TB	2.268	36.284
Top rebuild communication, PB	0.482	8.998
Top rebuild payload swap layers	318	370
ANDs per lookup, million	0.627	0.841

The AND row covers reads and rebuilding; the separately charged frontend adds a small amount. These are steady representation allocations, not peak process memory. Rebuilding also needs transient tag labels, saved secret controls, metadata planner workspace, and the $n/4$-slot parent residual collection. A copied main-array source or a second compaction buffer must be charged if the implementation does not operate in place. OT seeds, queues, GC streaming buffers, and extra replicas are additional. The model does not yet price storage I/O or claim that a whole state fits in one machine.

9.2 Mixed-protocol depth

GC metadata planning can be computed before payload routing, because the planning depends only on keys, flags, and tags. The packed word operations remain interactive: an OT layer's sender inputs depend on the payload shares produced by previous layers. Independent swaps in a layer batch together, but precomputing random OTs does not remove all online dependencies.

The primary top rebuild has 512-bucket butterfly depth $10h$ payload swap layers, plus source compactions, local extraction/refinement, and global stash collection. The model reports this serial-layer count; it is not a measured number of RTTs because OT preprocessing and transport scheduling can differ. Read payload products for all candidate slots run in parallel, so scanning a 64- or 128-slot leaf does not imply that many serial OT rounds. Duplicate propagation uses a parallel prefix rather than a linear chain in the new ledger.

9.3 Replicas and throughput

At 10,000 lookups/s, multiply the unframed MB/query by 10 to get aggregate GB/s across both communication directions. Directional provisioning must separate predominantly garbler-to-evaluator GC tables from the symmetric word-OT traffic. Aggregate bandwidth alone does not establish a state's batch service time or the replica count needed while the top rebuilds. For batch service time T_b, a steady-state group count must satisfy at least the relevant arrival/service constraint; long periodic rebuilds need an explicit availability schedule as well. Neither annual cloud cost nor CPU-limited throughput is claimed here.

9.4 An optional parent-local persistent stash

Instead of globally merging the residuals, one could retain each parent's 64-slot stash and search it with that parent's small bucket. The parent index is already determined by the disclosed small-bucket address, so this does not add address leakage if both addresses derive from the same true-or-dummy choice. It scans $z + 64$ slots and avoids the global collector, but adds $n/4$ persistent slots per table. Different children share a parent stash and must coordinate updates. Source admission must include those extra slots. This alternative is not included in the primary numbers, which follow the requested merged-residual organization.

10 Validation, remaining work, and recommendation

The cost program is a ledger of explicit kernels rather than a full 2PC implementation. Its checks cover binary-counter frequencies, exact ORCompact swap arithmetic, final-excess selection, conservation of keys, recursive leaf capacities, threshold boundary cases, and numerical tail formulas. Constructive simulations include empty, balanced, highly skewed, and random parents, plus residual counts 0, 1, 63, 64, and 65. Overflow conditions are tested before truncation.

Recommendation. Pursue the 512-to-64 design as the primary communication candidate, retaining 512-to-128 as a useful alternative when largest-table stash scanning or contention matters. The additional refinement and stash work is comfortably below the savings in ordinary 512-slot reads. Rebuilding again becomes the main amortized traffic term, as anticipated. No early rejection against the current Circuit OPRAM ledger is justified by these counts.

The following work remains before an implementation or stronger performance claim:

1. **Compile and count the actual circuits.** Replace the planner, scan, and parallel-propagation allowances by emitted-circuit counts; prove their label interfaces compose safely.
2. **Implement the packed backend.** Validate GC color-bit sharing, both OT directions, fresh masks/tweaks, scheduling, and application input/output boundaries. Benchmark local hashing and XOR work.
3. **Complete the security argument.** Prove adaptive lookup indistinguishability with absent tombstones, grouped bucket/stash updates, fresh epochs, and bounded failures. The sketch here is not a complete simulation proof.
4. **Certify probability calculations.** Check the analytic proof conditions and numerical witnesses, and choose a total fingerprint, PRF, and capacity failure budget for the intended service lifetime.
5. **Measure a distributed rebuild.** Account for metadata-label traffic inside each party, peak memory, SSD/RAM I/O, OT dependency layers, and the largest rebuild pause.
6. **Choose a deployment schedule.** Choose batch size and replica count from measured service time and latency targets; application writes additionally require replica consistency.

A Planner derivation and finite counts

For $a = 2^k$, a concrete ORCompact planner comprises balanced subtree popcounts, offset additions, shared threshold decoding, and leaf controls. Their AND allowances are respectively

$$2a - 2k - 2, \qquad a - k - 1, \qquad a(k-4) + 2k + 4, \qquad a/2.$$

Their sum is $P(a) = a(k - 1/2) - k + 1$. A shared decoder for all comparisons of a q-bit secret threshold against public indices uses $D(q) = 2^{q+1} - 2q - 2$ ANDs. This avoids charging a separate full comparator for every index.

Exact-padding marks first identify real-left entries and their total, then compute the needed dummy quota. Use *exclusive* dummy prefix ranks, which are at most $a - 1$, and select a dummy iff its rank is below the quota. Inclusive ranks can equal a and would require another bit. A work-efficient parallel prefix scan, comparisons, and gating give the $3ka + 4a - 2k - 2$ marking allowance. The construction assumes both real output counts fit; the probability argument or the preselection invariant establishes that condition.

For extracting a parent's residuals, $P(512) = 4344$ and $S(512) = 2304$. The additional 197 planner/check ANDs cover a nine-bit residual count, the 64 prefix occupancy flags, and the $r > 64$ test. At most $512 - z$ records can be residual, so nine bits suffice. The data copy into fixed stash positions is public; live flags preserve its secrecy.

The complete local kernel counts and per-level rebuild contributions are stored in the accompanying JSON. The model retains all global routing bits through the butterfly as a conservative choice; discarding consumed bits there offers a further optimization that is not included.

B Reproduction and source provenance

From the repository root, with Python, SciPy, and mpmath available:

```
python -m mpc_cost.hierarchical_split_cost
python -m unittest mpc_cost.test_hierarchical_split -v
python -m mpc_cost.render_hierarchical_split_report
```

Compile the delivered self-contained LaTeX source twice with pdfLaTeX. It embeds all numerical tables and bibliography entries and requires no project-specific input files or external figure files. The model writes `mpc_cost/results/hierarchical_split_comparison.json`; the report generator writes `output/latex/hierarchical_batch_oram_report.tex`.

Microrama supplies the hierarchical/butterfly design and the necessity of cached absences. ORCompact supplies the compaction network and exact swap counts. The mutable 2PC adaptation, exact-excess preselection, mixed payload interface, stash bounds, and numerical ledger are the present analysis. The Circuit OPRAM comparison uses the repository's 16 October 2026 interactive-shuffle ledger, whose traffic is 226.506916/286.945386 MB unframed and 237.832261/301.292656 MB with 5% framing. That ledger is itself a conditional circuit-composition model.

References

[1] Anonymous submission. *Microrama: Microsecond Oblivious Access to Gigabytes of Data*. User-supplied ongoing draft, `microrama.pdf`, especially Sections 2.1–2.4. The draft's native TEE mea-

surements are not used as 2PC performance measurements.

[2] Sajin Sasy, Aaron Johnson, and Ian Goldberg. *Fast Fully Oblivious Compaction and Shuffling*. CCS 2022, extended version. https://eprint.iacr.org/2022/1333. Sections 3 and 5.1, Appendix B.

[3] NIST. *Circuit Complexity: List of Circuits*. AES-128 with expanded key: 5120 ANDs; with key schedule: 6400 ANDs. https://csrc.nist.gov/projects/circuit-complexity/list-of-circuits. Accessed 17 October 2026.

[4] Samee Zahur, Mike Rosulek, and David Evans. *Two Halves Make a Whole: Reducing Data Transfer in Garbled Circuits Using Half Gates*. EUROCRYPT 2015. https://www.cs.virginia.edu/~evans/pubs/ec2015/.

[5] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. *Extending Oblivious Transfers Efficiently*. CRYPTO 2003. https://www.iacr.org/archive/crypto2003/27290145/27290145.pdf. The 160-byte word operation is this report's explicit chosen-message accounting, including a conventional 128-bit extension-matrix allowance.

[6] T.-H. Hubert Chan and Elaine Shi. *Circuit OPRAM: Unifying Statistically and Computationally Secure ORAMs and OPRAMs*. Cryptology ePrint Archive, 2016/1084, revised 2020. https://eprint.iacr.org/2016/1084. The comparison uses a repository compilation template rather than a performance number reported in this paper.